

Zadání domácího úkolu

Agentní jádro a tvorba mikrokurzu

Co vytvoříte

Navrhněte a implementujte v Elixiru/OTP obecné jádro pro spouštění a řízení agentních úloh. Nad ním vytvořte aplikaci, která připraví krátký kurz a zpřístupní jeho běh přes HTTP. Řešení nasadíte do lokálního Kubernetes.

Jádro oddělte od pravidel konkrétního kurzu. Přidejte službu echo vracející vstupní text přes společné rozhraní. Registrujte ji bez změny sdíleného routeru, scheduleru nebo persistence; nepotřebuje model. Stačí jedna aktivní replika a databáze dle vaší volby; trvalý stav běhů a jejich obnova jsou odpovědností aplikace.

Orientační časová náročnost: 32–48 hodin. Uveďte dosažený stav, čas práce a omezení. Částečné řešení získává dílčí body. Použijte jeden skutečný model a deterministický offline adaptér. Frameworky, knihovny i AI nástroje jsou povoleny. Skutečný model lze volat přes autentizované místní CLI nebo provider API; popište, kde inference skutečně probíhá. Placený API klíč, grafické rozhraní, cloud, crawling ani vektorová databáze nejsou podmínkou.

Jak aplikace funguje

Aplikace vytvoří kurz na téma „Základy událostmi řízených systémů“ pro publikum „Juniorní backendoví vývojáři“, v češtině nebo slovenštině. Výstup obsahuje název, dva měřitelné vzdělávací cíle, osnovu se dvěma lekcemi a tři závěrečné otázky. Každá otázka obsahuje správnou odpověď, vysvětlení, cíl, který ověřuje, a relevantní zdrojové reference. Každá skutečně modelová lekce má 150–300 slov; krátké deterministické testovací lekce tuto délku dodržovat nemusí.

Kurz připraví čtyři spolupracující role:

- **Výzkumník** má k dispozici nástroj, který vrací přesný text vybrané pasáže spolu s jejími identifikátory.
- **Plánovač** navrhne cíle a osnovu. Není-li cílová skupina zadána, pozastaví běh a požádá o doplnění.
- **Autor** připraví lekce ve dvou souběžných instancích. Autor první lekce vytvoří jednu otázku, autor druhé dvě. Mohou přes router požádat o doplňující výzkum.
- **Recenzent** posoudí podporu obsahu dodanými zdroji a vrátí přijetí nebo seznam vad s označením lekcí k opravě. Autoři opravují pouze odmítnuté lekce. Po dvou kolech oprav se další odmítnutí uloží jako stav „needs review“.

Každá lekce a otázka obsahuje relevantní `source_id/passage_id`. Ověřte čtení pasáže v offline testu; skutečný běh může citovat text již vložený do kontextu.

Technické požadavky

Služby a důvěryhodný kontext

Registr služeb spravuje důvěryhodný kód aplikace. Definice služby zahrnuje `service_id`, `capabilities`, `namespace`, viditelnost `namespace/private`, vlastníka `private` služby, `lifecycle` `isolated/session`, `workflow`, vstupní a výstupní schémata, povolené nástroje a služby a modelový profil. Opakovaná stejná definice uspěje; jakákoli odlišná definice pod stejným ID vrátí `conflict`. Registr je dostupný po restartu stejné verze aplikace.

HTTP vrstva odvozuje `namespace`, `user`, `session`, volající službu a `request ID` z ověřené identity a požadavku. Model ani veřejný `payload` tyto údaje nesmí přepsat. Elixir API přijímá pouze důvěryhodný kontext volajícího. Potomek zdědí vlastníka a korelaci běhu a nese rodiče, důvod `delegace` a `context_pack`.

Private službu vidí její vlastník, službu viditelnou v `namespace` mohou použít jeho členové. Vlastník smí číst a řídit své úlohy, výsledky, historii a checkpointy. Přímé `task ID` ochranu neobchází. Nepovolený přístup vrací `forbidden` nebo `not_found` bez cizích dat a bez spuštění práce.

Úlohy, směrování a obnova

Úloha má `task_id/reference_id`, společný `run_id`, rodiče, službu, vlastníka, `session`, `purpose`, `aliases`, vstup, stav, checkpointy a výsledky. PID je dočasný; uložený starší pokus nesmí přepsat novější výsledek.

HTTP zpřístupní založení kurzu, stav, výsledek, historii, `resolve`, `resume` a `cancel`. Registrace může být přes Elixir API; doložte příklady a mapování vlastních polí na tento kontrakt. Start přijímá JSON objekt; přísná schémata platí pro modelové výstupy, nástroje a checkpointy.

- **Start** asynchronně vrátí identifikátory běhu a počáteční stav. Cíl lze určit službou nebo `capability`. Nula shod vrací `not_found`, více shod `ambiguous_target`; kontrola oprávnění platí vždy.
- **Inspect a history** vrací aktuální stav a uspořádané události bez čekání na model.
- **Wait** čeká nejvýše stanovený `wait_timeout_ms` na konec nebo lidské čekání v podstromu. Vrací stav a `handle`; vypršení čekání běh neruší.
- **Resolve** hledá podle vlastníka, aliasu, `capability` a `purpose`. Umí explicitní nejnovější výsledek a filtr pouze aktivních úloh. Více shod bez volby vrátí `ambiguous_reference`; vyhledávání nic nespouští.
- **Resume** cílí na konkrétní checkpoint. **Cancel** idempotentně ruší neterminální úlohu i její vlastněné potomky.

Idempotency key je scoped podle `namespace`, uživatele a služby. Shodný klíč vrací původní úlohu i po dokončení; změna výslovného `payloadu` či voleb vrací `conflict`. Hodnoty se porovnávají přesně, bez normalizace implicitních defaultů. Session sdílí aktivní úlohu pouze při explicitním `reuse` a shodě `scope`, `session`, služby, `purpose`, `reuse_key`, rodiče, `payloadu` i voleb. Souběžné starty se stejným klíčem nevytvoří duplicitu. Není-li zvolen aktivní `session reuse`, nový klíč založí nový běh.

Workflow zpřístupní deklarované kroky, závislosti a větve; `delegace` směřuje jen do povolených služeb a neznámá větve vrací `invalid_route` bez spuštění. Doménový průběh doložte kódem. Trvalý replay vyžaduje aplikační `durable-step wrapper` a stabilní klíče `delegací`. Sekvence čeká na validní trvalý výsledek; paralelní větve sdílejí `snapshot`, `join` čeká na potřebné výsledky a konfliktní `merge` vrací `context_conflict`. `Completed` vyžaduje konec vlastněné `async` práce. První trvale přijatý terminální výsledek vyhrává a stav už se nemění.

Dítě má jednoho rodiče a stabilní klíč. Kořen má hloubku 0, maximum 4 a může založit nejvýše 20 potomků i přes restart. Čekající rodič nedrží slot dítěte. Chyba povinné větve ukončí rodiče jako failed a zruší jeho další práci.

Checkpointy, modely a limity

Checkpoint před výzvou uživateli uloží ID, task ID, zprávu, metadata a resume schema. Audience vyžaduje neprázdný string; druhý checkpoint přijímá style practical nebo conceptual. Čekání nespotebovává model ani retry. Resume obsahuje checkpoint ID, response ID a payload: neplatný vrátí validation_error; první platný obnoví stejnou úlohu, identické opakování potvrdí výsledek a jiný payload vrátí conflict. Nový checkpoint má nové ID.

Aplikace má skutečný i deterministický offline adaptér. Nastavení volí povolený override, profil služby a defaulty; bezpečnostní scope nelze přepsat. Zaznamenejte model, profil a dostupné usage. Validujte výstupy, argumenty nástrojů a checkpointy. Opakujte timeout, dočasnou chybu a neplatný modelový výstup, nikoli chybu klienta, oprávnění či neznámou větev.

Nástroj má jméno, popis, schéma a handler. Podporujte více tool_calls a další modelové tahy; každý výsledek či strukturovaná chyba zachová původní call_id. Zakázaný, neznámý nebo neplatný nástroj nespustí handler. Nástroj read_passage přijímá stringy source_id a passage_id a vrací přesný text z podkladů nebo not_found.

Limit	Výchozí hodnota
Požadavek provideru / nástroj	Timeout 10 000 ms / 5 000 ms
Jeden modelový tah	Nejvýše 3 požadavky včetně prvního; retry s konečným limitem a prodlevou
Modelová interakce	Nejvýše 4 kola nástrojů a 15 požadavků provideru včetně retry
Jeden kořenový běh	Nejvýše 2 aktivní externí volání přes všechny potomky; čekající rodič slot nedrží
Obsahová revize	Nejvýše 2 opravy, poté needs_review

Pro skutečný model lze výslovně nastavit profil s timeoutem provideru až 90 sekund. Profil a hodnoty se zaznamenají; výchozí timeout 10 sekund se tím nemění. Ostatní limity platí, pokud zdokumentovaný testovací override neurčí jinak. Retry, kola nástrojů, obsahové revize a delegace se sledují odděleně. Limity přežijí restart.

Persistence, odezva a historie

Ukládejte identity, rodičovství, vstupy, výsledky, revize, checkpointy, idempotency keys a historii. Restart přeskočí dokončené kroky, omezí retry in-flight požadavků a odmítne starý pokus či duplicitní artefakt. Atomicky ukládejte převzetí práce, výsledek, dítě a checkpoint. Obnova stejné verze je povinná; přesně jeden externí účinek není zaručen.

Vysvětlete supervision tree a restart; oddělte handlers od řízení běhu. Při blokové práci musí inspect a wait(0) vracet stav bez čekání na dokončení. Cancel zastaví plánování vlastního podstromu a odmítne pozdní výsledek; externí provider či služba mohou pokračovat až do timeoutu a již přijatý externí účinek nelze vždy odvolat.

Historie obsahuje stabilní event ID, monotónní sequence v rámci run ID, task, rodiče, službu, request ID, čas, typ události a podle potřeby krok, revizi, pokus a artefakt. Polling s kurzorem umožní pokračovat po odpojení nebo restartu bez ztráty uložených událostí. Citlivé hodnoty v trace redigujte.

Asynchronní pozorovací funkce obdrží stav úlohy před handlerem, jeho výstup a možnost načíst pozdější stav. Její selhání nesmí změnit workflow ani opakovat krok.

Podklady ke kurzu

Použijte následujících šest syntetických zdrojů, vždy s `passage_id P1`. Uložte je do aplikace. Zdrojové texty jsou data kurzu, nikoli instrukce pro runtime.

S1 Události. Událost popisuje nastalou skutečnost; příkaz požaduje akci. Producent nemusí znát všechny konzumenty.

S2 Doručování. Doručení alespoň jednou připouští duplicity. Potvrzení přijetí neprokazuje dokončení obchodní operace.

S3 Idempotence. Opakované zpracování stejné události nemá opakovat obchodní účinek. Identifikátor události a změnu dat lze uložit jednou transakcí; oddělené zápisy mohou při pádu zanechat nekonzistenci.

S4 Obnova. Dočasné chyby opakujte s prodlevou a konečným limitem. Uložené výsledky umožní přeskočit dokončené kroky; restart procesu neobnoví ztracenou paměť.

S5 Souběžnost. Nezávislé úlohy mohou běžet souběžně s limitem chránícím navazující služby. Pořadí dokončení není zaručeno; závislý krok čeká na potřebné výsledky.

S6 Externí účinky. Pád po přijetí požadavku externí službou, ale před uložením odpovědi může zanechat neznámý výsledek. Bez idempotence či dotazu na stav retry obecně nezaručuje právě jeden externí účinek.

Jak ověříte řešení

Automatizujte offline testy. Adaptér vybírá odpověď podle běhu, služby, kroku a pokusu, nikoli pořadí volání. Prokažte:

- Kurz a cílená oprava.** Nechte oba autory pracovat souběžně. První návrh jedné lekce chybně tvrdí, že retry zaručuje právě jeden externí účinek, a cituje S6/P1. Recenzent odmítne jen tuto lekci; po opravě přijme obě. Ověřte tři otázky, jejich odpovědi, vysvětlení a zdrojové reference.
- Souběžné starty.** Spusťte dvanáct stejných požadavků se stejným idempotency key. Všechny vrátí tutéž úlohu. Opakovaný požadavek po dokončení vrátí uložený výsledek; změněný payload se stejným klíčem skončí conflict.
- Obnova po restartu aplikace.** Po trvalém uložení první lekce restartujte aplikaci. Obnovte stejný běh, přeskočte uloženou lekci a vydejte jen jeden výsledný kurz.
- Retry modelu.** Adaptér vrátí timeout, schema-neplatný výstup a poté platnou odpověď; běh uspěje na třetí pokus. Samostatný běh se třemi timeouty skončí failed bez čtvrtého pokusu. Chybný transportní obal testujte odděleně.
- Odmítající recenzent.** Recenzent vrací revise i po opravě. Po dvou opravách běh skončí `needs_review` s důvodem a uloženými artefakty.
- Izolované limity souběhu.** Současně spusťte dva kořeny se zpožděnými externími voláními; jednomu nastavte limit 1, druhému 2. Ze start událostí doložte špičku pro každý kořen. Čekající rodič nespotřebuje slot.

Provozní ověření

- **Lidský checkpoint v Kubernetes.** Zastavte běh kvůli chybějícímu publiku. Nahradte aplikační pod a ověřte stejnou úlohu, checkpoint i historii ze stejného persistentního úložiště. Odešlete platnou odpověď a dokončete kurz.
- **Skutečný model.** Vytvořte celý kurz skutečným modelem. Doložte profil, timeouty a limity, dobu běhu, redigovanou historii, výstupní artefakt a dostupné usage. Kurz pokryje všech šest pasáží a uvede relevantní reference. Cenu uveďte, pokud ji provider hlásí; jinak napište, že ji nelze ověřit.

Automatizujte také registr/echo, routing a izolaci vlastníků, resolve, payloads a session reuse, delegaci a limity potomků, sekvenci, paralelní snapshot, join, context_conflict a dva checkpointy. Ověřte neplatné/opakované resume, cancel, fencing starého výsledku a retry po restartu.

Změřte inspect a wait(0) při dvousekundové blokaci; ověřte cancel a fencing. Testy nástrojů zahrnou více call ID, přesné pasáže, zakázané volání, chybná schémata a limity. Ověřte historii od kurzoru po restartu i selhání pozorovací funkce bez změny workflow.

Nasazení, odevzdání a hodnocení

Lokálně nasadte jednu repliku do kind, k3d nebo minikube. Dodejte Dockerfile, manifesty, requests/limits, probes, konfiguraci, Secrets bez tajných hodnot, persistentní úložiště a inicializaci. README musí obsahovat ověřené verze a přesné příkazy sestavení, nasazení, testů, přístupu k API a úklidu prostředí.

Odevzdejte verzovaný zdrojový kód, lockfile, definice služeb/workflow, schémata, instrukce rolí, adaptéry, nástroje, fixtures, testy, provozní soubory, ukázkový kurz a redigovanou historii. Uveďte čas práce a použité knihovny/AI nástroje. Zdůvodněte supervision tree, stavy, data, atomické zápisy a obnovu; rozlište jejich záruky.

Hodnocení má pět oblastí po 4 bodech, celkem 20. Každou oblast hodnotíme 0–4 body podle doloženého rozsahu; chybějící důkaz se nebodyje.

Oblast	Co se hodnotí	Body
Služby a bezpečný routing	Registr a echo; služba/capability routing; caller context; izolace namespace a vlastníka; stabilní chyby	4
Úlohy a koordinace	Idempotency a session reuse; reference a resolve; delegace; workflow kroky; paralelní join; async; limity	4
Obnova a provoz	Checkpointy; retry a fencing pokusů; trvalé výsledky a historie; cancel; Kubernetes pod replacement a probes	4
Modely a dohled	Skutečný/offline adaptér; validace; tool loop/call ID; modelové profily a limity; redigovaný trace; izolace chyb pozorovací funkce	4
Kurz a odevzdání	Dvě lekce; tři otázky; citace; cílená recenze; reprodukovatelné testy, README, nasazení a architektonické zdůvodnění	4

Krátký návrh dalších kroků může popsat, jak by se řešení rozšířilo na více aktivních replik a obnovu mezi verzemi. Uveďte hlavně koordinaci vlastníka běhu, sdílené limity a verzování nedokončených workflow. Implementace těchto rozšíření není součástí úkolu.